

shared memory multiprocessors & multicores

shared physical main memory

- **UMA** (“uniform memory access”): access to any word of physical main memory takes about the same amount of time
- **NUMA** (“nonuniform memory access”): main memory implemented in multiple components distributed among processors, accessing “close” memory is much faster
- **NUMA** systems **more scalable**, but also **more complex** for programmer trying to optimize performance

programmer can use **operations on shared memory locations** for thread communication, data sharing

... but often will need to *synchronize* accesses to shared data structures to avoid corruption

for **synchronization**, programmer can use constructs provided by programming language and/or libraries

... but how are these synchronization constructs implemented in machine/assembly language?

implementing “locks”

locks can be used to prevent concurrent accesses to a resource, i.e. provide **mutual exclusion**

- a thread wanting resource access must first *acquire* the lock that guards the resource (only one thread at a time allowed to hold the lock)
- *release* the lock when done

can implement a lock using a memory word in which record the lock state: **held (1)** or **free (0)**

to acquire lock, check if corresponding memory word contains 0, and if so, store a 1 in this word

... but must be done in one **atomic operation**

wouldn't work to use following code for acquiring lock:

```
la s0, lockvar
li t0, 1
acquire: lw t1, 0(s0)
         bnez t1, acquire
         sw t0, 0(s0)
```

problem here is that two threads, running on different cores, might read the contents of “lockvar” concurrently, both see a value of 0, and then both “acquire the lock” by writing 1!

1st implementation

uses the RISC-V instructions “**lr.w**” (“**load-reserved word**”) and “**sc.w**” (“**store-conditional word**”) (in extension `A`)

code for *acquiring* a lock:

```
        la s0, lockvar
        li t0, 1
try:    lr.w t1, 0(s0)
        bnez t1, try
        sc.w t2, t0, 0(s0)
        bnez t2, try
```

the **sc.w** instruction “**fails**” if there is a store (e.g. by a thread on some other core) to the memory word that occurs *between* the **lr.w** and the **sc.w** ... in this case, only outcome is that a non-zero value is written into destination register (here **t2**)

if **sc.w** instruction *doesn't fail*, contents of the register are stored into the memory location, and “**0**” is written into the destination register

to *release* the lock, write **0** into **lockvar**

... but there's a complication

above lock implementation assumes **sequential consistency** ... on systems with a weaker type of consistency, need to explicitly enforce ordering between instructions executed while holding lock, and those before/after

2nd implementation

uses the RISC-V instruction “**amoswap.w**” (in extension `A`)

code for *acquiring* a lock:

```
        la s0, lockvar
        li t0, 1
try:    lw t1, 0(s0)
        bnez t1, try
        amoswap.w.aq t1, t0, 0(s0)
        bnez t1, try
```

amoswap.w *atomically* (i) loads contents of a word into destination register, and (ii) stores value in second source register into the word

code for *releasing* a lock:

```
        amoswap.w.rl zero, zero, 0(s0)
```

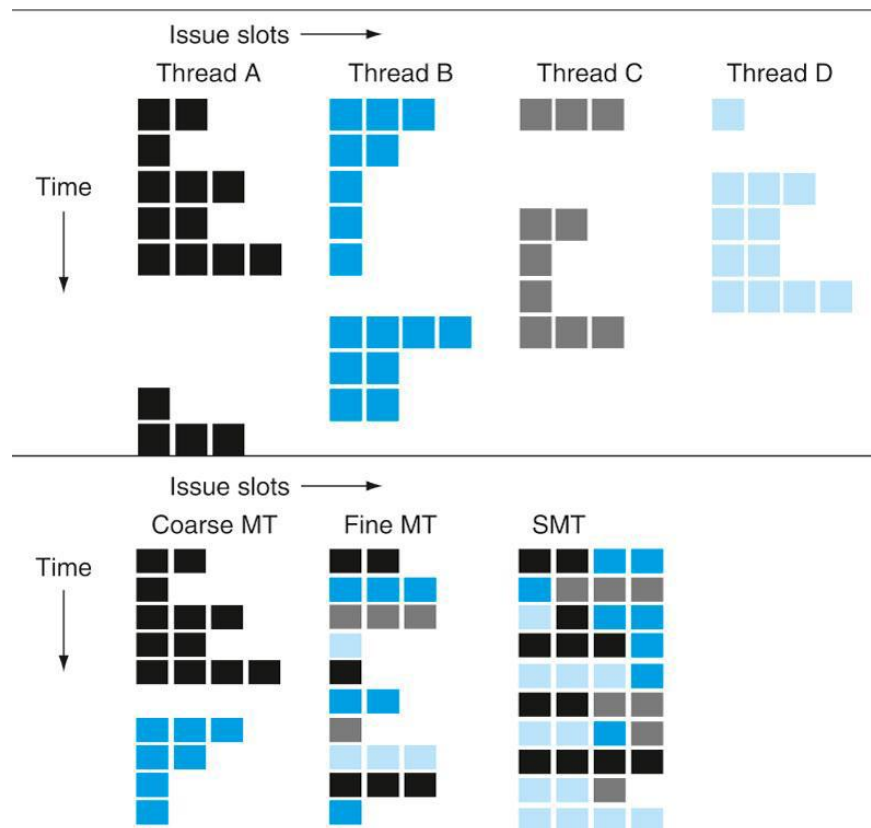
with the **aq**, **rl** suffixes, we get the needed instruction ordering guarantees

hardware multithreading

within a **single core**, replicate some components to **support execution of multiple threads**

three types:

- **coarse-grained multithreading**: when a thread stalls, switch to a different thread ... replicate hardware resources such as the **register file, PC** so that each of the threads alternating execution **have their own**
- **fine-grained multithreading**: switch every clock cycle!
- **simultaneous multithreading (called “hyper-threading”** by Intel): instructions from multiple (typically two) threads can be issued in the same clock cycle ... exploits techniques such as “**register renaming**”



(from text Fig. 6.5)